

VIT BHOPAL

Student Attendance Manager

Java Console Application

PROJECT REPORT

Project Title:	Student Attendance Manager
Project Type:	Java console application
Submitted To:	VITyarthi
Student Name:	Rachit Yadav
Register Number:	25BAI10669
Institution:	VIT BHOPAL UNIVERSITY

1. Introduction

Student Attendance Manager is a local Java application for maintaining student attendance records. A user can add students, list registered students, record a present or absent status for every student on a selected date, and view a date-specific or all-date attendance report. The program stores the records in a readable CSV file so attendance remains available after the application is closed.

The project demonstrates object-oriented programming using records, collections, file handling, date validation, exception handling, formatted console output, and percentage calculations. It is intentionally lightweight: it requires only the Java standard library and does not depend on a database, build tool, network connection, or external library.

2. Problem Statement

Attendance recorded informally can be difficult to preserve, update, and summarize. A teacher or class coordinator needs a simple way to answer questions such as:

- Which students are registered in the class?
- Who was present or absent on a particular date?
- How many times has each student attended?
- What is each student's attendance percentage across the saved records?

The application addresses this problem by combining a menu-driven console interface with persistent local storage and a summary report.

3. Functional Requirements

ID	Requirement	Description
FR-01	Add student	Save a unique student ID and non-empty student name.
FR-02	List students	Display all registered students with their IDs and names.
FR-03	Record attendance	Save Present or Absent status for every student on a valid date.
FR-04	View report	Display present count, absent count, and attendance rate for each student.
FR-05	Date filtering	Show a report for one date or for all saved attendance dates.
FR-06	Persistent storage	Load existing records at startup and save changes to a CSV file.
FR-07	Input validation	Reject blank fields, invalid dates, and invalid attendance choices.
FR-08	Exit safely	Save the current data before closing the application.

The application presents these requirements through the following menu:

- Add student
- List students
- Record attendance
- View attendance report
- Save and exit

4. Non-functional Requirements

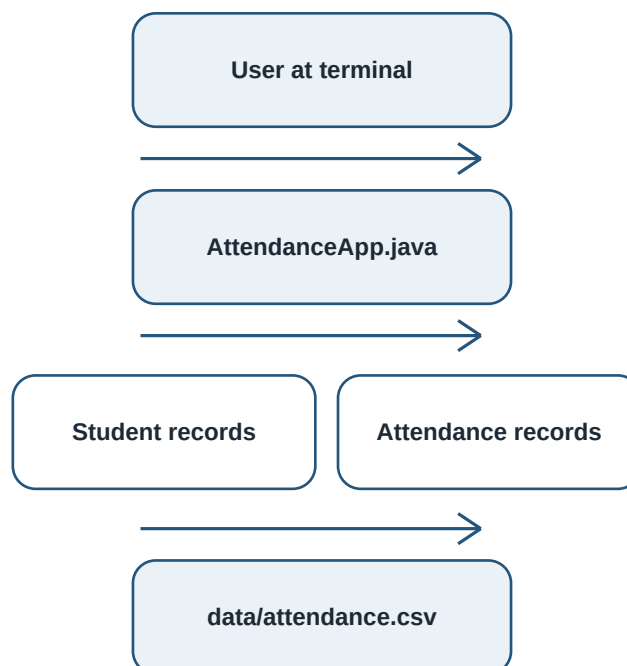
- Usability: A numbered menu and plain-language prompts make the application suitable for a first-time console user.
- Reliability: The data directory and CSV file are created automatically when data is first saved.
- Validation: Student IDs and names cannot be blank; dates must be valid ISO dates; attendance choices must be P or A.
- Performance: LinkedHashMap provides efficient lookup by student and attendance key for a small classroom record set.
- Maintainability: The application keeps input handling, storage, reporting, and data models in one clearly organized source file for this basic project scope.
- Portability: The program uses standard Java APIs and requires Java 16 or newer because it uses records and modern switch syntax.
- Privacy: Attendance data is stored locally and is not transmitted over a network.

5. System Architecture

The application follows a compact layered structure:

- Presentation layer: AttendanceApp.java displays the menu, reads input, and prints results.
- Domain model: Student represents an ID and name; AttendanceStatus represents a date, student ID, and present/absent value.
- Attendance service: The application manages student and attendance maps, handles date filtering, and calculates totals.
- Persistence layer: data/attendance.csv stores the saved records and is loaded at startup.
- Reporting layer: The report operation calculates present count, absent count, and attendance percentage.

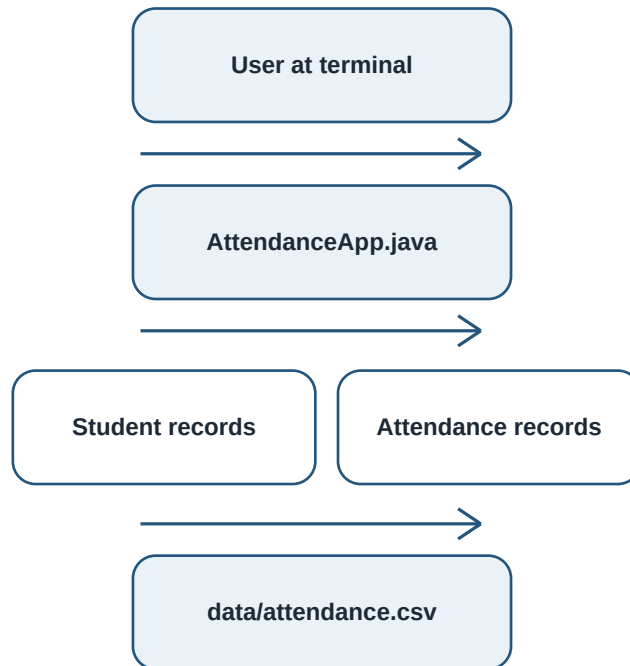
Data flow



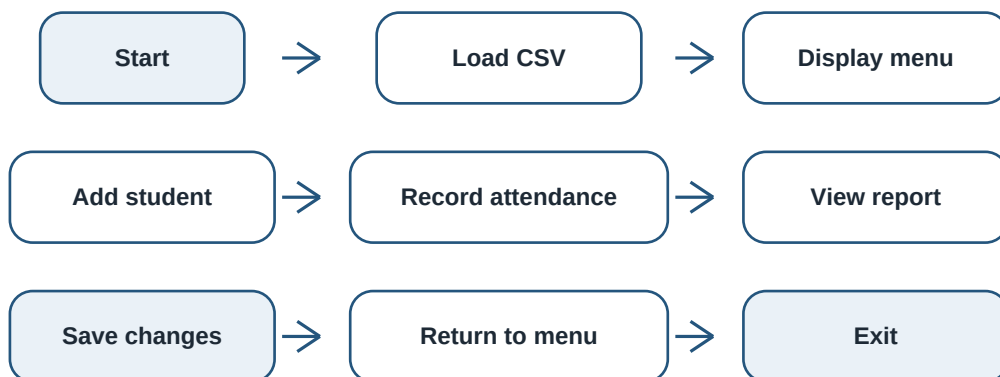
At startup, the CSV file is read if it exists. Student and attendance records are kept in memory while the program runs. Adding a student or recording attendance immediately rewrites the CSV file, and the exit option saves once more before termination.

6. Design Diagrams

System architecture



Application workflow



The workflow returns to the main menu after each operation, allowing repeated attendance entry and reporting during one session.

7. Design Decisions & Rationale

- Console interface: Keeps the application lightweight, easy to run, and easy to demonstrate during evaluation.
- CSV storage: Provides persistence without database installation or JDBC configuration. The saved records remain readable and easy to inspect.
- LinkedHashMap: Preserves insertion order for predictable student listing while supporting direct lookup by ID.
- Composite attendance key: The date and student ID together identify one attendance record and allow a later entry for the same date to update the earlier value.
- Immediate saving: Writes changes after adding a student and after recording attendance, reducing the chance of losing an update.
- Input loops: Invalid dates and attendance choices are rejected and requested again instead of terminating the application.
- Percentage calculation: Present count divided by total recorded attendance, displayed to one decimal place; a student with no records displays a dash.

8. Implementation Details

8.1 Student and attendance models

The Student record stores an ID and name. The AttendanceStatus record stores the date, student ID, and a boolean present value. These records provide compact immutable data objects and make the calculations readable.

8.2 Attendance persistence

The application stores one record per line using the following four-field format:

```
date,studentId,studentName,status
```

For example, the verified demonstration record currently stored by the application is:

```
2026-09-18,25BAI10669,RACHIT YADAV,P
```

When the file is loaded, malformed rows are skipped so that one damaged line does not prevent normal startup. When data changes, the complete current collection is written back to the file.

8.3 Input validation

Dates are parsed with `LocalDate.parse` and must use the `YYYY-MM-DD` format. Blank student IDs and names are rejected. During attendance entry, the program accepts only `P` for Present or `A` for Absent and repeats the prompt for any other value.

8.4 Report calculation

For each student, the report scans the saved attendance records, applies the optional date filter, counts present and absent entries, and calculates the percentage using $\text{present} / \text{total} * 100$.

9. Screenshots & Results

The following console captures are text-based evidence from live runs of the application. They are included as readable report output rather than invented GUI screenshots.

9.1 Startup and menu

```
=== Student Attendance Manager === Attendance data: data/attendance.csv 1. Add student 2. List students 3. Record attendance 4. View attendance report 5. Save and exit
```

9.2 Recording attendance

```
Recording attendance for 2026-09-18 25bai10375 - rishit (P=Present, A=Absent): P Attendance saved for 2026-09-18.
```

9.3 Report output

```
Attendance report ID Name Present Absent Rate
----- 25bai10375 rishit 1 0
100.0%
```

9.4 Persistence result

A second Java process was started after the first process exited. The application loaded the saved student and attendance record from data/attendance.csv and displayed the same 100.0% report. This confirms persistence across program runs.

10. Testing Approach

Test ID	Scenario	Expected result	Status
TC-01	Compile source	No compiler errors	Passed
TC-02	Add student	Student is accepted and saved	Passed
TC-03	Record P/A values	Status is saved for selected date	Passed
TC-04	View date report	Counts and rate are correct	Passed
TC-05	Fresh-process reload	Saved records are loaded	Passed
TC-06	Invalid date	Prompt repeats with validation message	Implemented
TC-07	Invalid attendance choice	Prompt repeats until P or A	Implemented

The compile check used javac, followed by scripted console sessions. The first session added a student and recorded attendance; a second fresh process verified that the data was read from disk. The project intentionally has no build tool or external test library.

11. Challenges Faced

- Choosing a simple storage format that remains readable while preserving the association between a date and a student.
- Keeping attendance available between program runs without introducing a database or external dependency.
- Preventing duplicate attendance rows for the same date and student by using a composite key.
- Handling invalid dates and attendance choices without terminating the program.
- Calculating individual attendance rates from the same stored records without duplicating data.

12. Conclusion

Student Attendance Manager meets the core requirements of a basic attendance system through a focused Java console application. It provides student registration, date-based attendance entry, persistent CSV storage, and attendance reporting with percentage calculations. The implementation demonstrates practical use of Java records, maps, file I/O, date parsing, input validation, and formatted console output.

13. Future Enhancements

- Add teacher or administrator login support.
- Support editing or deleting a student and correcting attendance records.
- Add monthly and date-range reports with class-level attendance statistics.
- Export formatted reports to CSV or PDF from inside the application.
- Replace CSV storage with a database when the number of students and records grows.
- Add automated unit tests and a graphical or web-based interface.

14. References

- Oracle Java Documentation - <https://docs.oracle.com/en/java/>
- Oracle Java I/O Tutorial - <https://docs.oracle.com/javase/tutorial/essential/io/>
- Oracle Java Collections Tutorial - <https://docs.oracle.com/javase/tutorial/collections/>
- Oracle Java Date-Time Tutorial - <https://docs.oracle.com/javase/tutorial/datetime/>